

친구 - 풀이

작성자: 박선재

부분문제 1

$N \leq 3000$ 이므로, $1 \leq i, j \leq N$ 이고 $i \neq j$ 인 모든 순서쌍 (i, j) 에 대해 i 번 학생과 j 번 학생이 친구인지 알 수 있으므로, 모든 학생의 친구 수를 계산할 수 있다.

부분문제 2

학생들을 집의 좌표 순으로 정렬한다. 좌표가 모두 다른 정수이고 $K_1, K_2 \leq 10$ 이므로, 어떤 학생과 친구가 될 수 있는 학생은 정렬된 순서에서 양옆으로 각각 최대 10명뿐이다. 따라서 각 학생마다 양옆 10명씩만 확인하면 친구의 수를 정확하게 계산할 수 있다.

부분문제 3

좌표가 1 이상 N 이하의 정수이므로 좌표를 인덱스로 하는 배열을 쓸 수 있다. 학교별로 좌표에 학생 수를 기록한 누적합 배열을 만들면, 각 학생에 대해 자신과 같은 학교에 다니면서 거리가 K_1 이하인 학생의 수, 다른 학교에 다니면서 거리가 K_2 이하인 학생의 수를 빠르게 계산할 수 있다.

부분문제 4

모든 학생이 같은 학교 소속이므로, 각 학생의 친구 수는 자기 자신을 제외하고 집 사이의 거리가 K_1 이하인 학생의 수와 같다. 모든 학생의 집 위치를 정렬하고, 이분 탐색을 사용하면 각 학생의 친구 수를 빠르게 계산할 수 있다.

부분문제 5

모든 학생이 2개의 학교 중 하나에 다니므로, 각 학교에 대해 해당 학교에 다니는 학생들의 집 좌표들을 따로 모은 후, 각각 정렬한다. 이후 부분문제 4와 비슷하게 이분 탐색을 사용하면 각 학생의 친구 수를 빠르게 계산할 수 있다.

부분문제 6

자신과 다른 학교를 다니면서 집의 거리가 K_2 이하인 학생 수는, 전체 학생 중에서 집의 거리가 K_2 이하인 학생 수에서 같은 학교를 다니면서 집의 거리가 K_2 이하인 학생 수를 뺀 것과 같다는 사실을 관찰한다. 전체 학생의 집 좌표들을 정렬한 배열과, 각 학교에 대해 해당 학교에 다니는 학생들의 집 좌표들을 따로 모아 정렬한 배열을 준비한 후, 이분 탐색을 사용하면 각 학생의 친구 수를 빠르게 계산할 수 있다.

수열 정렬하기 - 풀이

작성자: 장근영

기본 관찰

한 번의 시행에서 기준값 x 를 고르면, 수열은 $[A_i \leq x]$ 와 $[A_i > x]$ 를 이어 붙인 형태가 된다. 여기서 각각의 부분수열 안에서는 원래 수열에서의 상대적인 순서가 유지된다.

여러 번의 시행에서 선택한 기준값들을 생각하자.

먼저 같은 기준값을 여러 번 선택하는 것은 의미가 없다. 한 번 x 를 기준으로 나누고 나면, 이미 모든 x 이하의 원소가 모든 x 초과인 원소보다 앞에 있으므로, 다시 같은 x 를 선택해도 수열은 변하지 않는다.

또한 서로 다른 두 기준값 x, y 에 대해, 이 둘을 적용하는 순서도 중요하지 않다. $x < y$ 라고 하자.

먼저 x 를 적용한 뒤 y 를 적용하면 다음과 같다. x 를 적용한 뒤 수열은 $[A_i \leq x] + [A_i > x]$ 가 된다. 이제 y 를 적용하면, 앞쪽의 $[A_i \leq x]$ 에 속한 원소들은 모두 y 이하이므로 그대로 앞에 남는다. 뒤쪽의 $[A_i > x]$ 중에서는 $x < A_i \leq y$ 인 원소들이 그다음으로 오고, $y < A_i$ 인 원소들이 마지막에 온다. 따라서 최종 수열은 $[A_i \leq x] + [x < A_i \leq y] + [y < A_i]$ 가 된다.

반대로 먼저 y 를 적용한 뒤 x 를 적용해도 같은 결과가 나온다. y 를 적용한 뒤 수열은 $[A_i \leq y] + [A_i > y]$ 가 된다. 이제 x 를 적용하면, 앞쪽의 $[A_i \leq y]$ 안에서 $A_i \leq x$ 인 원소들이 먼저 오고, 그다음에 $x < A_i \leq y$ 인 원소들이 온다. 뒤쪽의 $[A_i > y]$ 에 속한 원소들은 모두 x 초과이므로 마지막에 남는다. 따라서 이 경우에도 최종 수열은 $[A_i \leq x] + [x < A_i \leq y] + [y < A_i]$ 가 된다.

각 단계에서 같은 범위에 속한 원소들끼리는 원래 수열에서의 상대적인 순서가 계속 유지된다. 따라서 두 기준값 x, y 를 적용한 결과는 적용 순서와 무관하다.

결국 여러 번의 시행에서 선택한 기준값들은 순서를 바꾸어도 최종 결과가 변하지 않는다. 따라서 선택한 기준값들을 중복 없이 정렬해서 $x_1 < x_2 < \dots < x_k$ 라고 두어도 된다.

이제 이 기준값들을 오름차순으로 적용한다고 생각하자. 처음 x_1 을 적용하면 수열은 $[A_i \leq x_1] + [x_1 < A_i]$ 가 된다. 그다음 x_2 를 적용하면 첫 번째 부분 $[A_i \leq x_1]$ 은 모두 x_2 이하이므로 그대로 앞에 남고, 뒤쪽 부분 $[x_1 < A_i]$ 이 $[x_1 < A_i \leq x_2]$ 와 $[x_2 < A_i]$ 로 나뉜다.

이 과정을 반복하면, 최종 수열은 $[A_i \leq x_1], [x_1 < A_i \leq x_2], \dots, [x_{k-1} < A_i \leq x_k], [x_k < A_i]$ 를 차례대로 이어 붙인 수열이 된다.

즉, 시행들의 순서는 중요하지 않고, 선택한 기준값들의 집합만 중요하다. 따라서 문제는 값의 범위 $[1, N]$ 을 몇 개의 연속한 구간으로 나누어서, 각 구간에 속한 값들만 원래 순서대로 뽑은 부분수열이 비내림차순이 되게 만드는 문제로 볼 수 있다.

값의 구간 수가 c 개라면, 그 구간들을 만들기 위해 필요한 시행 횟수는 $c - 1$ 번이다.

부분문제 1

모든 원소가 1 또는 2이다.

이미 모든 1이 모든 2보다 앞에 있다면 수열은 비내림차순이므로 답은 0이다.

그렇지 않다면 어떤 2가 어떤 1보다 앞에 있다. 이 경우 $x = 1$ 로 한 번 시행하면 모든 1이 앞으로 가고, 모든 2가 뒤로 가므로 수열이 정렬된다.

따라서 수열에 2가 1보다 앞에 등장하는 경우가 있으면 답은 1, 그렇지 않으면 답은 0이다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이다.

부분문제 2

$N \leq 15$ 이다.

의미 있는 기준값은 $1, 2, \dots, N - 1$ 뿐이다. 따라서 가능한 기준값의 부분집합을 모두 시도할 수 있다.

각 부분집합 S 에 대해, S 의 원소들을 오름차순으로 적용한 결과를 직접 만들어 본다. 그 결과가 비내림차순이면, $|S|$ 가 가능한 시행 횟수이다.

모든 부분집합을 확인하면서 가능한 $|S|$ 의 최솟값을 구하면 된다.

부분집합의 개수는 최대 2^{N-1} 개이고, 각 경우에 대해 시행을 직접 시뮬레이션하면 충분하다.

전체 시간 복잡도는 $\mathcal{O}(2^N N^2)$ 이다.

부분문제 3

$N \leq 100$ 이다.

값의 구간 $[l, r]$ 에 대해, 값이 l 이상 r 이하인 원소들만 원래 순서대로 뽑았을 때 그 부분수열이 비내림차순인지 확인하자. 이를 $\text{ok}(l, r)$ 라고 하자.

$\text{ok}(l, r)$ 를 가장 단순하게 계산하려면, 모든 위치 쌍 $p < q$ 를 확인하면 된다.

만약 $l \leq A_p \leq r, l \leq A_q \leq r$ 이면서 $A_p > A_q$ 인 쌍이 존재한다면, 구간 $[l, r]$ 만으로 이루어진 부분수열은 비내림차순이 될 수 없다. 그런 쌍이 없다면 가능하다.

하나의 (l, r) 에 대해 모든 위치 쌍을 확인하는 데 $\mathcal{O}(N^2)$ 시간이 걸린다. 가능한 (l, r) 쌍은 $\mathcal{O}(N^2)$ 개이므로, 모든 $\text{ok}(l, r)$ 를 계산하는 데 $\mathcal{O}(N^4)$ 시간이 걸린다.

이제 DP를 하자. $\text{dp}[i]$ 를 값 $1, 2, \dots, i$ 를 몇 개의 연속한 값 구간으로 나누어야 하는지의 최솟값이라고 하자.

초기값은 $\text{dp}[0] = 0$ 이다.

마지막 구간이 $[j + 1, i]$ 라고 하면, 이 구간은 $\text{ok}(j + 1, i)$ 가 참이어야 한다. 따라서 $0 \leq j < i$ 인 모든 j 중에서 가능한 것을 골라 $\text{dp}[i] = \min(\text{dp}[j] + 1)$ 로 계산할 수 있다.

최소 시행 횟수는 구간 수보다 1 작으므로, 정답은 $\text{dp}[N] - 1$ 이다.

전체 시간 복잡도는 $\mathcal{O}(N^4)$ 이다.

부분문제 4

$N \leq 750$ 이다.

부분문제 3과 같은 DP를 사용하되, $ok(l, r)$ 를 더 빠르게 계산한다.

값의 구간 $[l, r]$ 에 대해, 수열 A 를 왼쪽에서 오른쪽으로 한 번 훑는다. 이때 $l \leq A_i \leq r$ 인 원소만 보면서, 직전에 선택된 값보다 현재 값이 작아지는 순간이 있는지 확인한다.

현재까지 선택한 마지막 값을 **last**라고 하자. $l \leq A_i \leq r$ 인 원소를 만났을 때 **last** $> A_i$ 라면, $ok(l, r)$ 는 거짓이다. 그렇지 않다면 **last**를 A_i 로 갱신한다.

이렇게 하면 하나의 (l, r) 에 대해 $ok(l, r)$ 를 $\mathcal{O}(N)$ 시간에 계산할 수 있다.

가능한 (l, r) 쌍은 $\mathcal{O}(N^2)$ 개이므로, 모든 $ok(l, r)$ 를 계산하는 데 $\mathcal{O}(N^3)$ 시간이 걸린다.

이후 DP는 부분문제 3과 동일하다. $dp[0] = 0$ 으로 두고, 마지막 구간 $[j + 1, i]$ 가 가능한 모든 j 에 대해 $dp[i] = \min(dp[j] + 1)$ 을 계산한다.

정답은 $dp[N] - 1$ 이다.

전체 시간 복잡도는 $\mathcal{O}(N^3)$ 이다.

부분문제 5

모든 원소가 서로 다르다.

수열의 길이가 N 이고, 모든 값이 1 이상 N 이하이므로, 수열 A 는 $1, 2, \dots, N$ 의 순열이다.

값 i 가 등장하는 위치를 p_i 라고 하자.

값 i 와 $i + 1$ 이 같은 값 구간에 들어간다고 하자. 같은 구간 안에서는 원래 수열에서의 상대적인 순서가 유지된다.

따라서 $p_i > p_{i+1}$ 이면, 원래 수열에서 $i + 1$ 이 i 보다 앞에 있으므로 i 와 $i + 1$ 을 같은 구간에 둘 수 없다. 즉, 반드시 i 와 $i + 1$ 사이를 나누는 기준값을 골라야 한다.

반대로 $p_i < p_{i+1}$ 이면, i 와 $i + 1$ 사이를 나누지 않아도 이 둘의 순서는 올바르다.

따라서 답은 $p_i > p_{i+1}$ 인 i 의 개수이다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이다.

부분문제 6

이제 추가 제약이 없다.

등장하는 서로 다른 값들을 오름차순으로 $v_1 < v_2 < \dots < v_M$ 이라고 하자.

각 값 v_i 에 대해, L_i 를 v_i 가 처음 등장하는 위치, R_i 를 v_i 가 마지막으로 등장하는 위치라고 하자.

인접한 두 등장값 v_i, v_{i+1} 를 생각하자.

만약 $R_i > L_{i+1}$ 이라면, 어떤 v_i 가 어떤 v_{i+1} 보다 뒤에 등장한다는 뜻이다. 즉, 원래 수열 안에 v_{i+1}, v_i 순서로 등장하는 두 원소가 존재한다.

이 둘이 같은 값 구간에 남아 있다면, 구간 내부의 상대적인 순서는 바뀌지 않으므로 비내림차순이 될 수 없다. 따라서 이 경우에는 반드시 v_i 와 v_{i+1} 사이를 나누어야 한다.

이때 기준값 x 는 $v_i \leq x < v_{i+1}$ 가 되도록 고르면 된다. 예를 들어 $x = v_i$ 를 고를 수 있다.

반대로 $R_i < L_{i+1}$ 이라면, 모든 v_i 가 모든 v_{i+1} 보다 앞에 등장한다. 따라서 v_i 와 v_{i+1} 를 같은 구간에 두어도 이 둘 사이의 순서는 문제가 없다.

그러므로 반드시 나누어야 하는 경계는 정확히 $R_i > L_{i+1}$ 인 경계들이다.

이 경계들을 모두 나누면, 각 값 구간 안에서는 인접한 등장값들 v_i, v_{i+1} 가 모두 $R_i < L_{i+1}$ 을 만족한다. 따라서 구간 안의 값들은 원래 수열에서도 작은 값부터 큰 값 순서로 등장하고, 그 구간의 부분수열은 비내림차순이다.

구간들 자체도 값의 범위가 증가하는 순서로 이어 붙여지므로, 전체 수열 역시 비내림차순이 된다.

따라서 정답은 $R_i > L_{i+1}$ 인 i 의 개수이다.

구현은 다음과 같이 하면 된다.

1. 각 값 x 에 대해 첫 등장 위치 $\text{first}[x]$, 마지막 등장 위치 $\text{last}[x]$ 를 계산한다.
2. 값 $1, 2, \dots, N$ 을 순서대로 보면서 실제로 등장한 값만 확인한다.
3. 직전에 등장한 값이 p , 현재 등장한 값이 x 일 때, $\text{last}[p] > \text{first}[x]$ 이면 답을 1 증가시킨다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이고, 메모리 복잡도는 $\mathcal{O}(N)$ 이다.

산책로 - 풀이

작성자: 윤교준

정점과 간선에 양의 가중치가 있는 트리가 주어졌을 때, 서로 다른 두 정점 사이를 잇는 경로의 가중치를 최대화해야 한다. 여기서, 경로 P 의 가중치 $w(P)$ 는 "경로에 속한 간선의 가중치의 총합 $s(P)$ "에서 "경로에 속한 정점의 가중치의 최댓값 $m(P)$ "을 뺀 값으로 정의된다.

부분문제 1

가능한 모든 $\frac{N(N-1)}{2}$ 가지의 경로 각각에 대해, 그 경로의 가중치를 선형에 계산할 수 있다.

전체 시간 복잡도는 $\mathcal{O}(N^3)$ 이다.

부분문제 2

경로의 한쪽 끝 정점을 트리의 루트로 고정하자. 한 번의 DFS로, 각 정점에 대해 루트와 그 정점 사이를 잇는 경로의 "간선 가중치 총합"과 "정점 가중치의 최댓값"을 모두 계산할 수 있다. 즉, 한쪽 끝 정점이 고정되어 있을 때, 가능한 모든 $N - 1$ 가지의 경로의 가중치를 선형에 알 수 있다.

이 작업을 모든 정점에 대해 반복하면, 전체 시간 복잡도 $\mathcal{O}(N^2)$ 에 답을 구할 수 있다.

부분문제 3

트리의 지름이 항상 답이 된다. 이는 선형 시간에 알아낼 수 있다.

부분문제 4

주어지는 트리가 선형이다.

일반성을 잃지 않고, 정점의 가중치 A_* 가 모두 서로 다르다고 가정하자. 이는 충분히 작은 양의 실수 ε 을 정한 후, 각 A_i 에 $i\varepsilon$ 만큼 더했다고 생각하면 된다.

고정된 v 에 대해, $m(P_v) = A_v$ 를 만족하는 경로 $P_v = \langle s_v \rightarrow s_v + 1 \rightarrow \dots \rightarrow e_v \rangle$ 의 가중치를 최대화하기 위해서는

- $s_v - 1$ 의 값은 $l < v$ 이고 $A_l > A_v$ 이면서 가장 큰 l 과 같다. (편의상, $A_0 := +\infty$.)
- $e_v + 1$ 의 값은 $r > v$ 이고 $A_r > A_v$ 이면서 가장 작은 r 과 같다. (편의상, $A_{N+1} := +\infty$.)

스택을 사용하여 왼쪽에서 오른쪽으로 배열 A_* 을 순회하면, 각 v 에 대해 그에 대응되는 s_v 를 모두 선형 시간에 계산할 수 있다. e_v 에 대해서도 대칭적으로 계산할 수 있다.

따라서, 최적의 경로의 후보가 되는 N 개의 경로 P_* 를 얻을 수 있다. 각 경로의 $s(P_*)$ 값은 배열 L_* 의 누적합을 통해 상수 시간에 계산할 수 있다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이다.

부분문제 5

주어지는 트리가 정점 N 을 중심으로 별 모양을 이룬다. 따라서, 하나의 경로에는 간선이 최대 두 개 존재한다.

하나의 간선으로 구성된 경로들은 자명하다.

일반성을 잃지 않고, $A_1 < A_2 < \dots < A_{N-1}$ 라고 가정하자. 이는 정점 N 을 제외한 나머지 $N - 1$ 개의 정점을 A_* 의 값에 따라 정렬하여 reindexing했다고 생각하면 된다.

두 개의 간선으로 구성된 경로는 항상 $P := \langle x \rightarrow N \rightarrow v \rangle$ ($x < v < N$) 꼴이다. 이때, v 를 고정하면

$$w(P) = L_x + (L_v - \max\{A_v, A_N\})$$

이므로, L_x 가 최대가 되는 $x < v$ 를 선택하는 것이 최적이다.

전체 시간 복잡도는 $\mathcal{O}(N \log N)$ 이다.

부분문제 6

다음 핵심 관찰이 필요하다: 상수 X 를 고정하고, $A_* > X$ 인 정점들을 제거하자. 남아있는 연결 컴포넌트 트리들의 지름을 구해, 그중에서 지름이 최대가 되는 경로를 P_X 라고 하자. $s(P_X) - X$ 를 최대화하는 X 의 값이 X_{opt} 라면, $P_{X_{\text{opt}}}$ 는 문제의 답이 된다.

$m(P_X) \leq X$ 이므로, 문제의 답이 되는 경로 P_{opt} 에 대해, $X = m(P_{\text{opt}})$ 일 때에 $s(P_X) - X$ 도 최대가 된다. 또한, $X \in \{A_1, A_2, \dots, A_N\}$ 만 고려해도 충분하다.

이 부분문제에서는 고려해야 하는 X 의 값이 최대 20가지 이므로, 각 X 에 대해 직접 포레스트를 구성하여 지름을 계산하면 충분하다.

전체 시간 복잡도는 $\mathcal{O}(20 \times N)$ 이다.

부분문제 7

부분문제 6의 핵심 관찰을 적용하자.

X 의 값이 점점 커지면, 포레스트에는 점점 정점과 간선이 추가되면서, 두 트리가 하나의 간선에 의해 하나의 연결된 트리으로 합쳐지는 과정이 반복된다.

두 트리 T_1, T_2 가 간선 e 에 의해 하나의 트리 $T := T_1 \cup \{e\} \cup T_2$ 로 합쳐졌다고 하자. T_1 과 T_2 의 지름이 각각 $P_1 := \langle s_1 \rightarrow \dots \rightarrow e_1 \rangle$, $P_2 := \langle s_2 \rightarrow \dots \rightarrow e_2 \rangle$ 이라고 하자. 트리의 지름의 성질에 의해, T 의 지름은 다음 6개의 경로 중 하나이다:

- 경로 P_1
- 경로 P_2
- 한쪽 끝 정점은 s_1 이거나 e_1 이고, 반대쪽 끝 정점은 s_2 이거나 e_2 인 4개의 경로

따라서, 트리에서 임의의 두 정점 사이의 거리를 효율적으로 계산할 수 있다면, 두 트리가 간선에 의해 하나로 합쳐져서 생기는 새로운 트리의 지름도 효율적으로 계산할 수 있다. 정점 사이의 거리는 최소 공통 조상 알고리즘을 통해 계산하면 충분하다.

즉, A_* 의 값이 작은 정점부터 차례대로 추가하면서, 각 연결 컴포넌트 트리의 지름에 대한 정보를 관리하면, 모든 $X \in \{A_1, A_2, \dots, A_N\}$ 에 대해 $s(P_X)$ 의 값을 효율적으로 계산할 수 있다.

전체 시간 복잡도는 $\mathcal{O}(N \log N)$ 이다.